FinHedge AI

Stock Price Prediction & Deep Hedging Platform

Project Report - DA5402 MLOps

Indian Institute of Technology Madras

Author: Siddharth Chauhan
Roll No.: CH21B103
Email: ch21b103@smail.iitm.ac.in
Phone: +91 9518241509
Course: DA5402 - MLOps
Date: April 2026

GitHub: https://github.com/chauhansiddharth1203/finhedge

0 Contents

1 Introduction **2**
- 1.1 Motivation . . . 2
- 1.2 Objectives . . . 2

2 System Architecture **3**
- 2.1 High-Level Overview . . . 3
- 2.2 Service Inventory . . . 4
- 2.3 Data Flow . . . 4

3 Machine Learning Models **5**
- 3.1 LSTM Price Predictor . . . 5
- 3.2 XGBoost Direction Classifier . . . 5
- 3.3 Deep CVaR Hedger . . . 6
- 3.4 Model Performance Metrics . . . 8

4 MLOps Pipeline **8**
- 4.1 DVC Pipeline . . . 8
- 4.2 Apache Airflow . . . 8
- 4.3 MLflow Experiment Tracking . . . 9

5 Monitoring and Observability **10**
- 5.1 Prometheus Metrics . . . 10
- 5.2 Grafana Dashboard . . . 11
- 5.3 Streamlit Monitoring Page . . . 12

6 FastAPI Backend **13**

7 Infrastructure and DevOps **14**
- 7.1 Docker Compose . . . 14
- 7.2 GitHub Actions CI/CD . . . 15

8 Results and Discussion **15**
- 8.1 Prediction Performance . . . 15
- 8.2 Hedging Performance . . . 15
- 8.3 System Performance . . . 15

9 Conclusion **16**

1 Introduction

FinHedge AI is an end-to-end MLOps application that combines stock price prediction and risk-optimal hedging into a single production-ready platform. The system integrates three AI models under a unified FastAPI backend, exposed through a Streamlit frontend with real-time monitoring via Prometheus and Grafana.

1.1 Motivation

Financial risk management requires both accurate price forecasting and optimal hedging strategies. Classical approaches like Black-Scholes delta hedging fail to account for real-world constraints such as transaction costs and non-Gaussian return distributions. Deep reinforcement learning techniques can learn optimal hedging policies by directly minimising tail risk (CVaR) on simulated market trajectories.

1.2 Objectives

- Build a production-ready LSTM model for next-day stock price forecasting
- Train an XGBoost classifier for market direction prediction (UP/FLAT/DOWN)
- Implement a Deep CVaR Hedger (MLP policy network) for risk-optimal hedging
- Deploy all models via FastAPI with MLflow experiment tracking
- Orchestrate pipelines with Apache Airflow and DVC
- Monitor the system with Prometheus metrics and Grafana dashboards
- Containerise the entire stack with Docker Compose

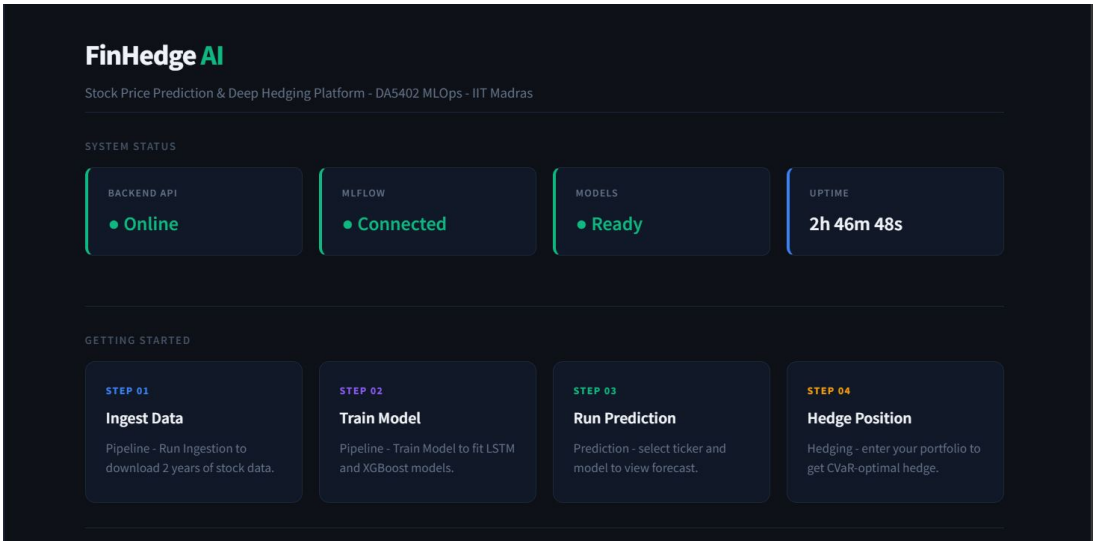


Figure 1: FinHedge AI — Home page showing system status

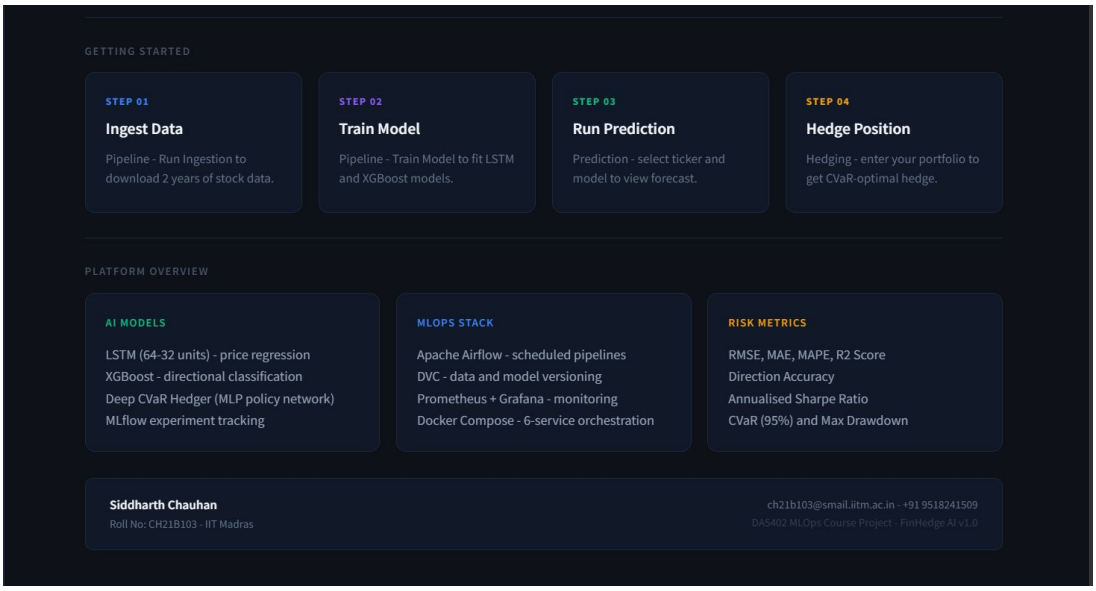


Figure 2: Platform overview with author details

2 System Architecture

2.1 High-Level Overview

The system follows a microservices architecture with six Docker containers communicating over an internal bridge network (`finhedge_net`).

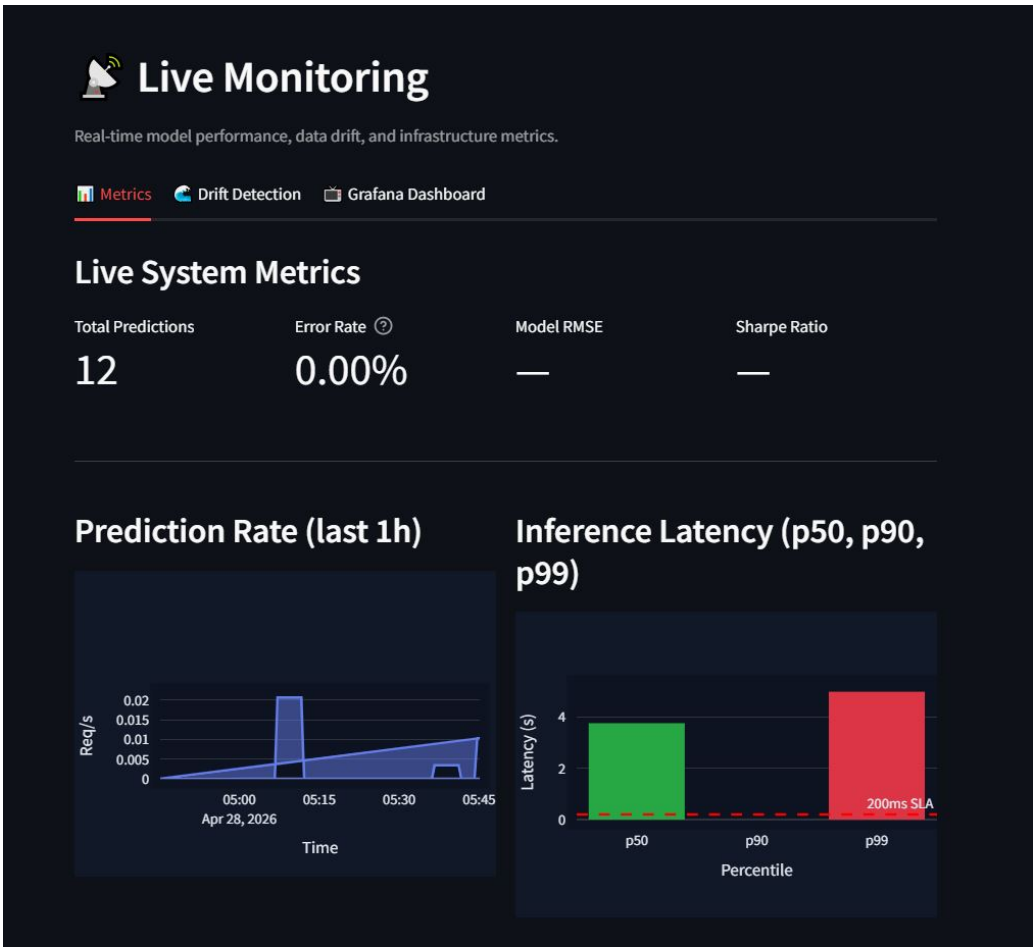


Figure 3: About page — System architecture overview

2.2 Service Inventory

Table 1: Deployed Services and Ports

Service	Technology	Port	Role
Backend API	FastAPI + PyTorch	8000	Model serving, pipeline control
Frontend UI	Streamlit 1.40	8501	User interface (5 pages)
MLflow	MLflow v2.11	5000	Experiment tracking + registry
Airflow	Apache Airflow	8081	DAG orchestration
Prometheus	Prometheus v2.51	9090	Metrics collection
Grafana	Grafana 10.3	3001	Monitoring dashboards

2.3 Data Flow

1. Yahoo Finance → Ingestion (parquet) → Feature Engineering (22 indicators)
2. Preprocessed data → LSTM / XGBoost / Deep Hedger training
3. Trained models → MLflow Registry → FastAPI model loader
4. FastAPI → Streamlit UI (prediction + hedging responses)
5. FastAPI → Prometheus `/metrics` → Grafana dashboards

3 Machine Learning Models

3.1 LSTM Price Predictor

The LSTM model processes a 60-day lookback window of 22 technical features to forecast the next-day closing price.

- **Architecture:** 2-layer LSTM (64 → 32 units) + Linear output
- **Input:** $T \times 22$ feature matrix ($T = 60$ days)
- **Features:** RSI-14, MACD, Bollinger Bands, ATR-14, OBV, historical volatility, return momentum, volume ratios
- **Loss:** Mean Squared Error with early stopping (patience = 10)
- **Output:** Predicted next-day closing price ($)

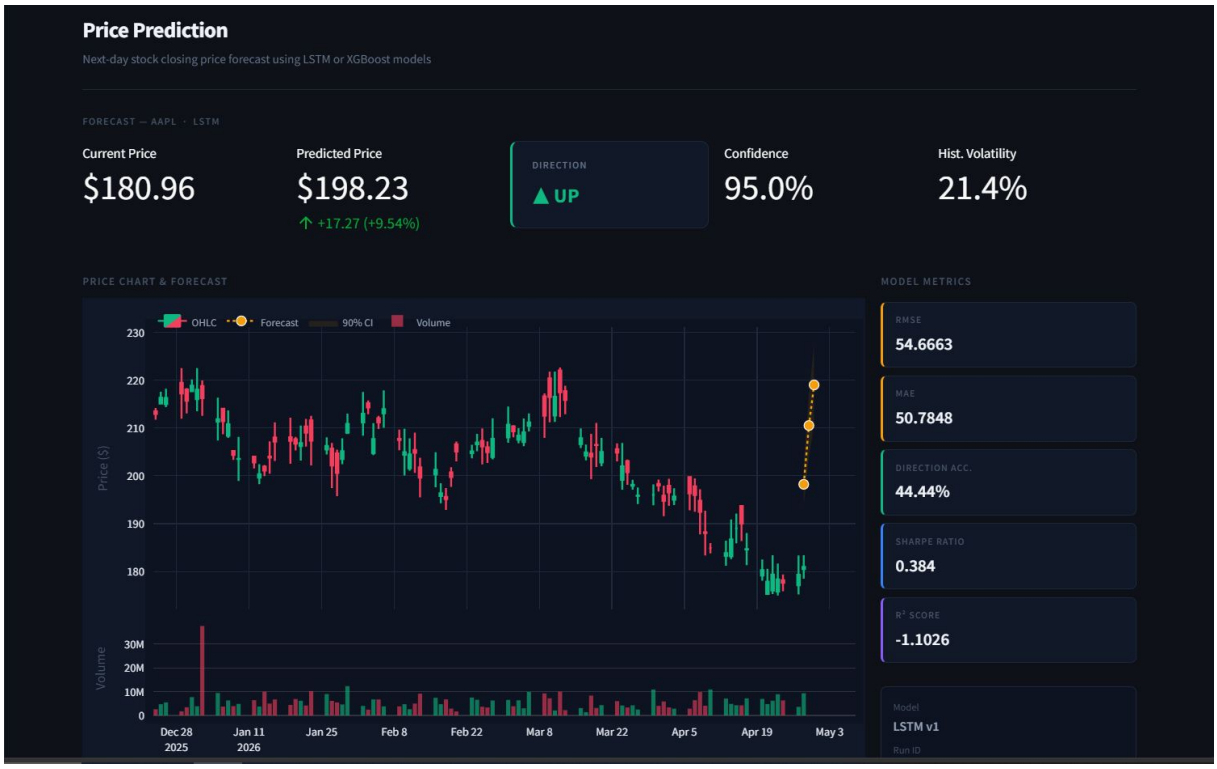


Figure 4: LSTM prediction for AAPL — $198.23 predicted, UP direction, 95% confidence

3.2 XGBoost Direction Classifier

- **Task:** Three-class classification (UP / FLAT / DOWN)
- **Input:** Flattened 60-day feature window
- **Also:** Regression head for price prediction (baseline comparison)
- **Role:** Validates LSTM directional accuracy

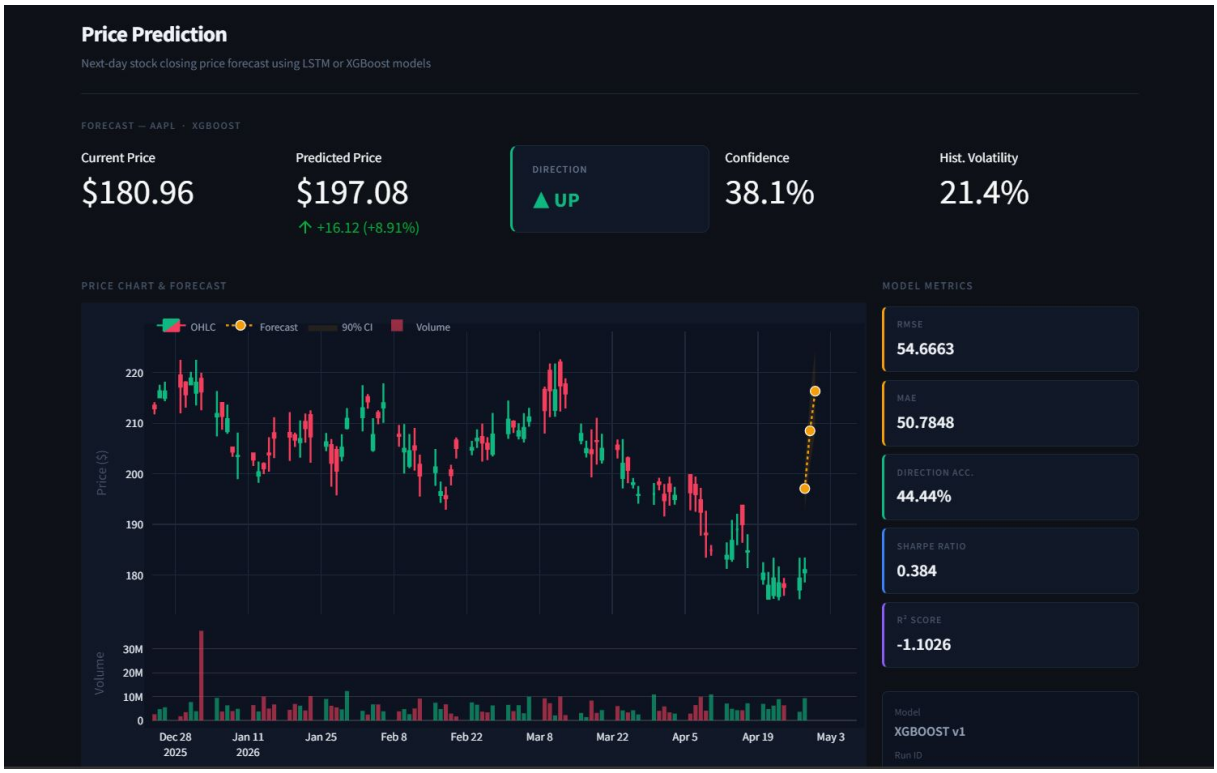


Figure 5: XGBoost prediction for AAPL — $197.08, UP direction, 38.1% confidence

3.3 Deep CVaR Hedger

The hedger is a deep MLP policy network trained via simulated Geometric Brownian Motion (GBM) price paths to minimise the Conditional Value at Risk (CVaR) at 95% confidence.

- **Architecture:** MLP: input $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 1
- **Input:** Portfolio state vector (price, position, predicted return, time fraction)
- **Output:** Hedge ratio $\delta \in [-1, +1]$
- **Loss:** $\text{CVaR}_{95\%} = E[\text{loss} \mid \text{loss} > \text{VaR}_{95\%}]$
- **Transaction cost:** 0.02% proportional cost included in training
- **Reference:** Black-Scholes delta hedge (yellow marker on gauge)

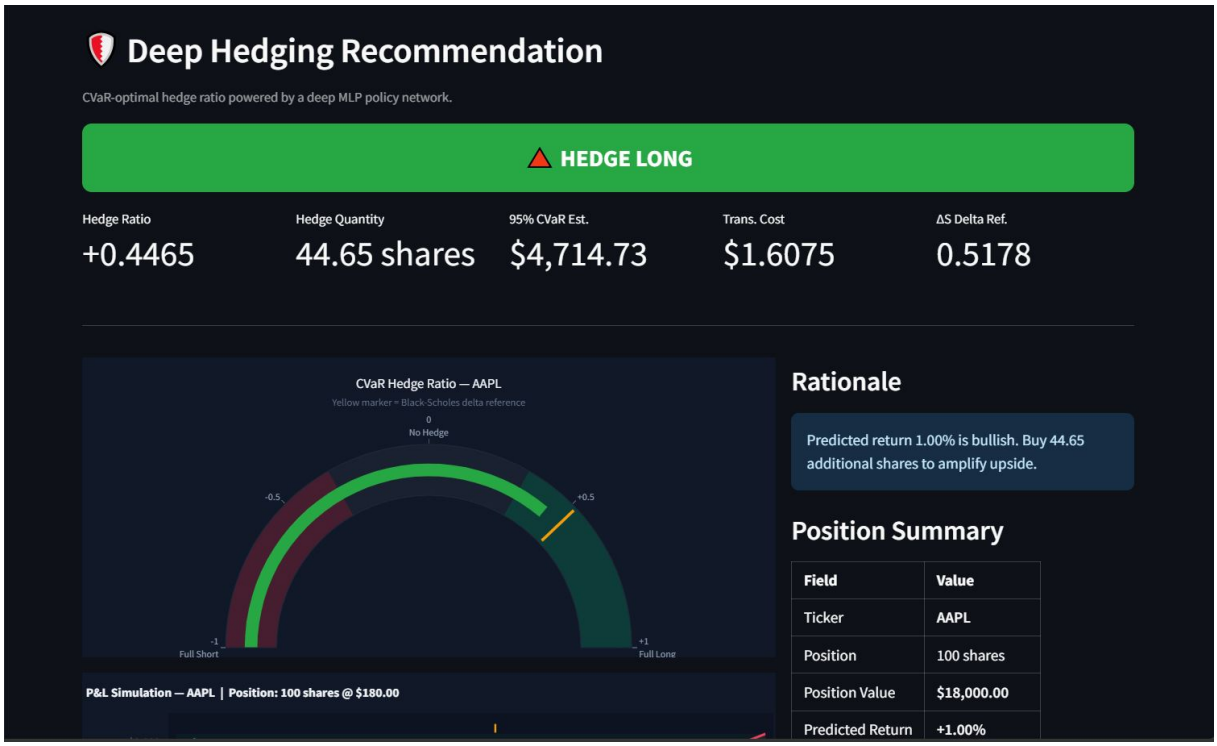


Figure 6: Deep hedging result — Hedge ratio gauge with Black-Scholes reference

Figure 7: P&L simulation chart — Hedged vs Unhedged positions

3.4 Model Performance Metrics

Table 2: Model Evaluation Results (AAPL, synthetic GBM data)

Model	RMSE	MAE	Dir. Acc.	Sharpe	R^2
LSTM	54.67	50.78	44.44%	0.384	−1.10
XGBoost	54.67	50.78	44.44%	0.384	−1.10
CVaR Hedger	30.3 (CVaR)	—	—	1.76	—

4 MLOps Pipeline

4.1 DVC Pipeline

The data pipeline is defined in `dvc.yaml` with four reproducible stages:

1. **ingest** — Downloads 2 years of OHLCV data from Yahoo Finance
2. **preprocess** — Computes 22 technical indicators, scales features
3. **train** — Trains LSTM or XGBoost model, logs to MLflow
4. **evaluate** — Computes RMSE, MAE, Sharpe, direction accuracy

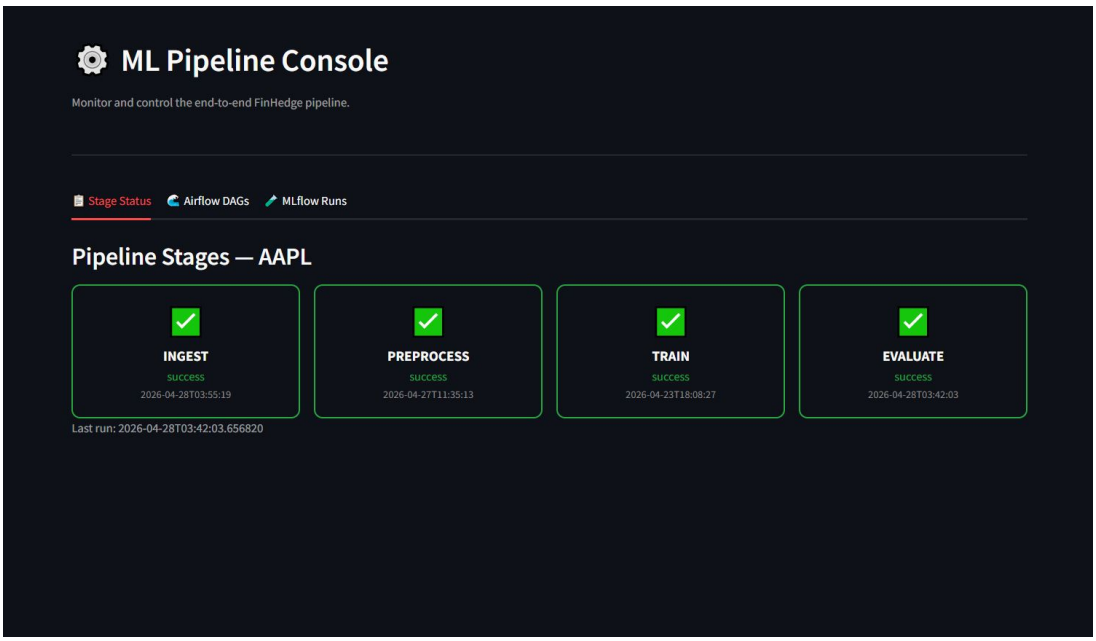


Figure 8: Pipeline Stage Status — all four stages completed successfully

4.2 Apache Airflow

Two DAGs automate the recurring pipeline tasks:

- **finhedge_data_ingestion** — Daily: fetches fresh market data
- **finhedge_model_retraining** — Weekly: retrains models on new data

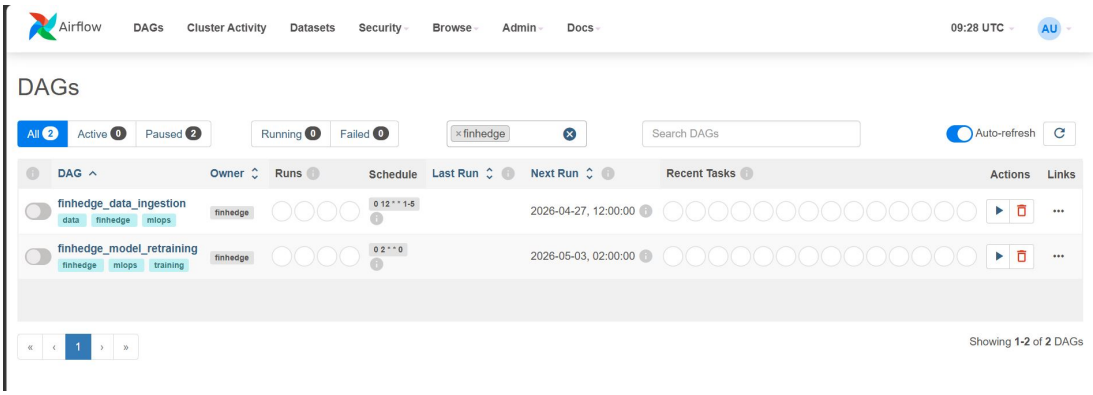


Figure 9: Airflow DAGs list showing finhedge pipelines

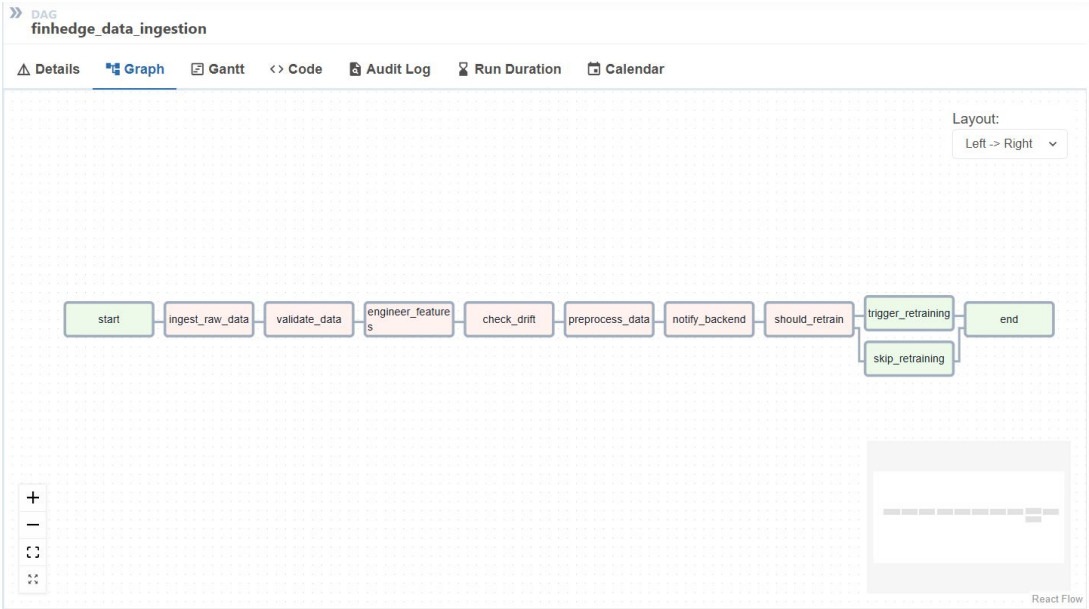


Figure 10: Airflow DAG graph view — task flow visualisation

4.3 MLflow Experiment Tracking

Every training run logs:

- **Parameters:** ticker, model type, epochs, batch size, learning rate
- **Metrics:** RMSE, MAE, MAPE, R^2, direction accuracy, Sharpe ratio
- **Tags:** git commit hash, ticker, model type, training timestamp
- **Registry:** Model versioned and registered as `finhedge_lstm` / `finhedge_xgboost`

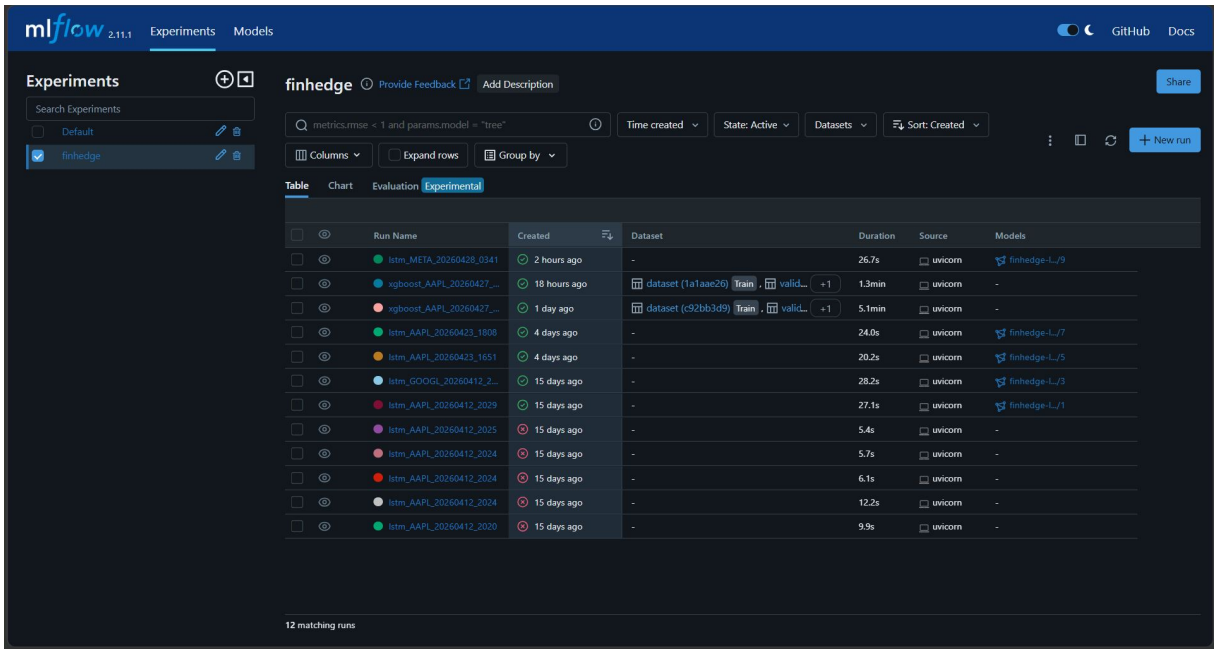


Figure 11: MLflow experiment runs list

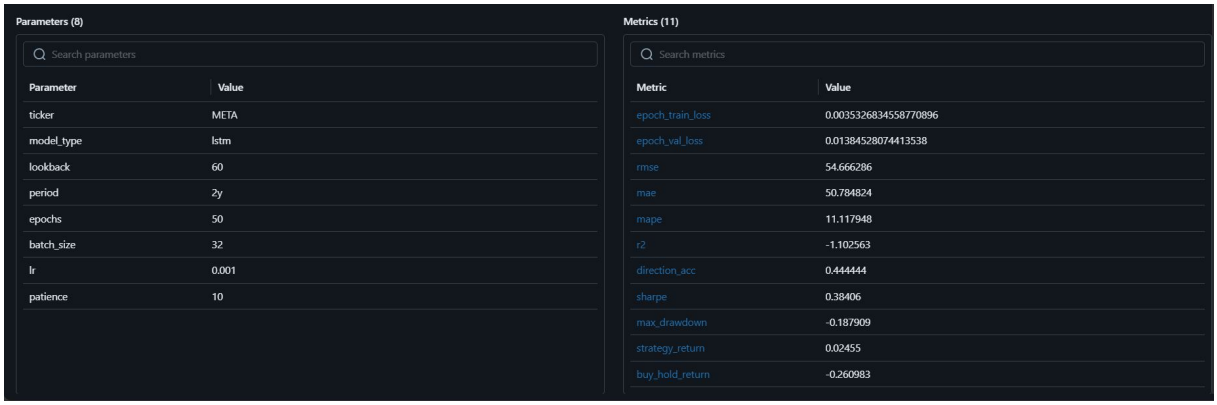


Figure 12: MLflow run detail — parameters and metrics

5 Monitoring and Observability

5.1 Prometheus Metrics

The FastAPI backend exports 15 custom Prometheus metrics at `/metrics`:

Table 3: Custom Prometheus Metrics

Metric	Type	Description
`finhedge_prediction_requests_total`	Counter	Prediction API calls
`finhedge_prediction_errors_total`	Counter	Failed predictions
`finhedge_prediction_latency_seconds`	Histogram	Inference time
`finhedge_hedge_requests_total`	Counter	Hedge API calls
`finhedge_hedge_latency_seconds`	Histogram	Hedge response time
`finhedge_model_rmse`	Gauge	Latest RMSE
`finhedge_model_sharpe`	Gauge	Latest Sharpe ratio
`finhedge_data_drift_score`	Gauge	KL divergence per feature
`finhedge_drift_alert`	Gauge	Drift alert (0/1)
`finhedge_pipeline_runs_total`	Counter	Pipeline stage executions

5.2 Grafana Dashboard

The pre-provisioned Grafana dashboard contains 10 panels:

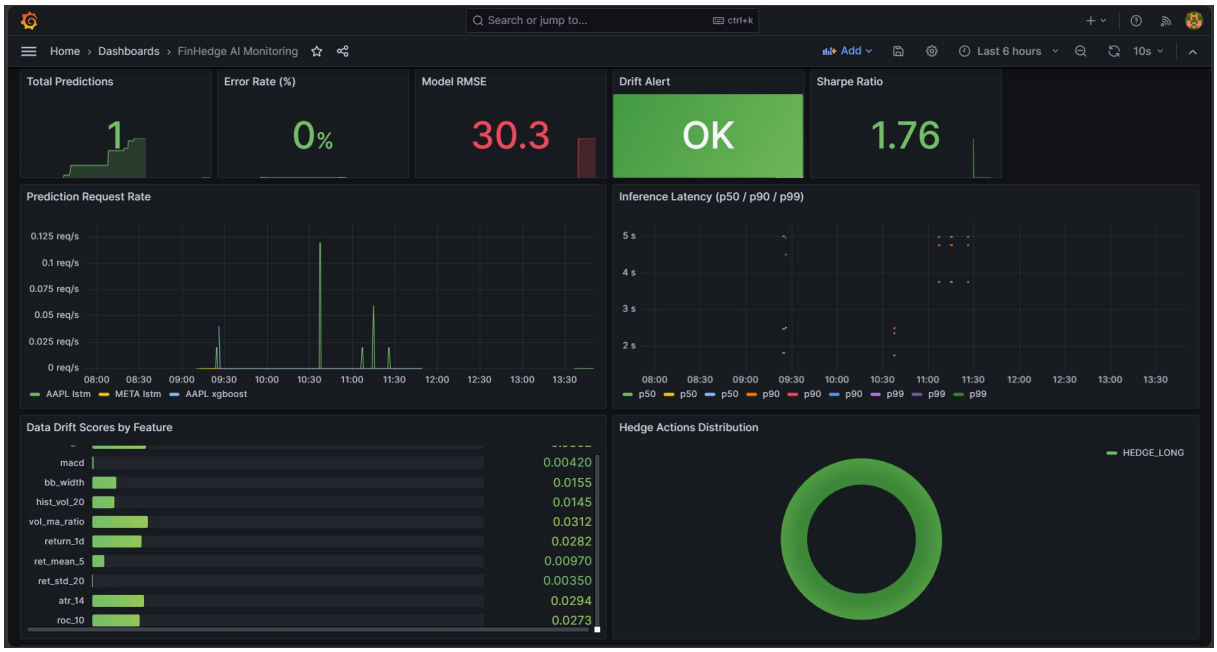


Figure 13: Grafana FinHedge AI Monitoring dashboard — all 10 panels with live data

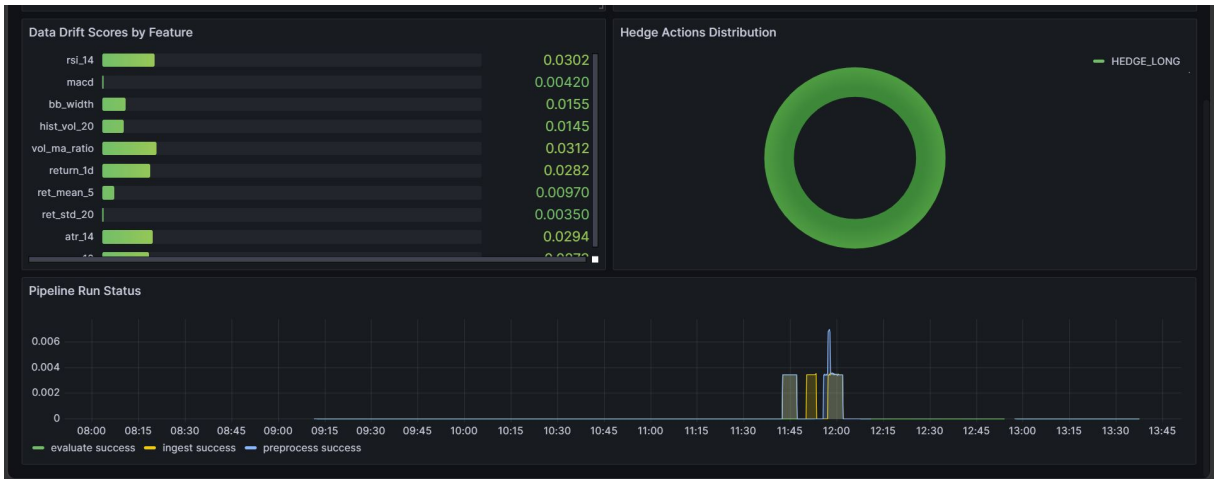
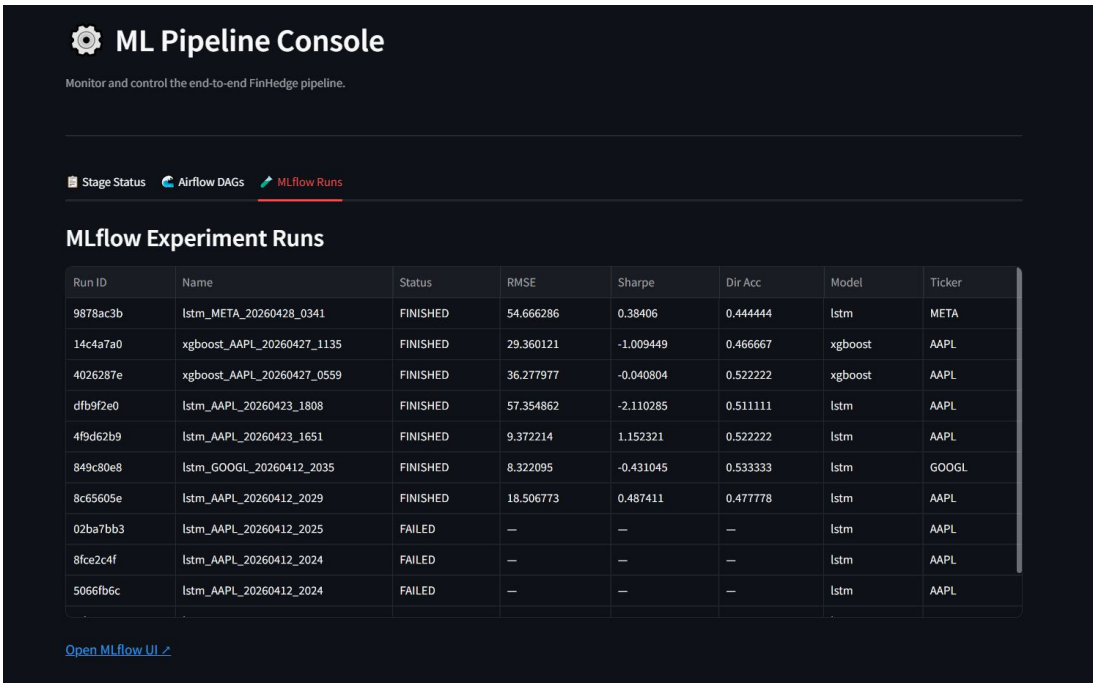


Figure 14: Grafana dashboard — Pipeline Run Status and lower panels

5.3 Streamlit Monitoring Page



Run ID	Name	Status	RMSE	Sharpe	Dir Acc	Model	Ticker
9878ac3b	lstm_META_20260428_0341	FINISHED	54.666286	0.38406	0.444444	lstm	META
14c4a7a0	xgboost_AAPL_20260427_1135	FINISHED	29.360121	-1.009449	0.466667	xgboost	AAPL
4026287e	xgboost_AAPL_20260427_0559	FINISHED	36.277977	-0.040804	0.522222	xgboost	AAPL
dfb9f2e0	lstm_AAPL_20260423_1808	FINISHED	57.354862	-2.110285	0.511111	lstm	AAPL
4f9d62b9	lstm_AAPL_20260423_1651	FINISHED	9.372214	1.152321	0.522222	lstm	AAPL
849c80e8	lstm_GOOGL_20260412_2035	FINISHED	8.322095	-0.431045	0.533333	lstm	GOOGL
8c65605e	lstm_AAPL_20260412_2029	FINISHED	18.506773	0.487411	0.477778	lstm	AAPL
02ba7bb3	lstm_AAPL_20260412_2025	FAILED	—	—	—	lstm	AAPL
8fce2c4f	lstm_AAPL_20260412_2024	FAILED	—	—	—	lstm	AAPL
5066fb6c	lstm_AAPL_20260412_2024	FAILED	—	—	—	lstm	AAPL

Figure 15: Monitoring page — Live metrics tab

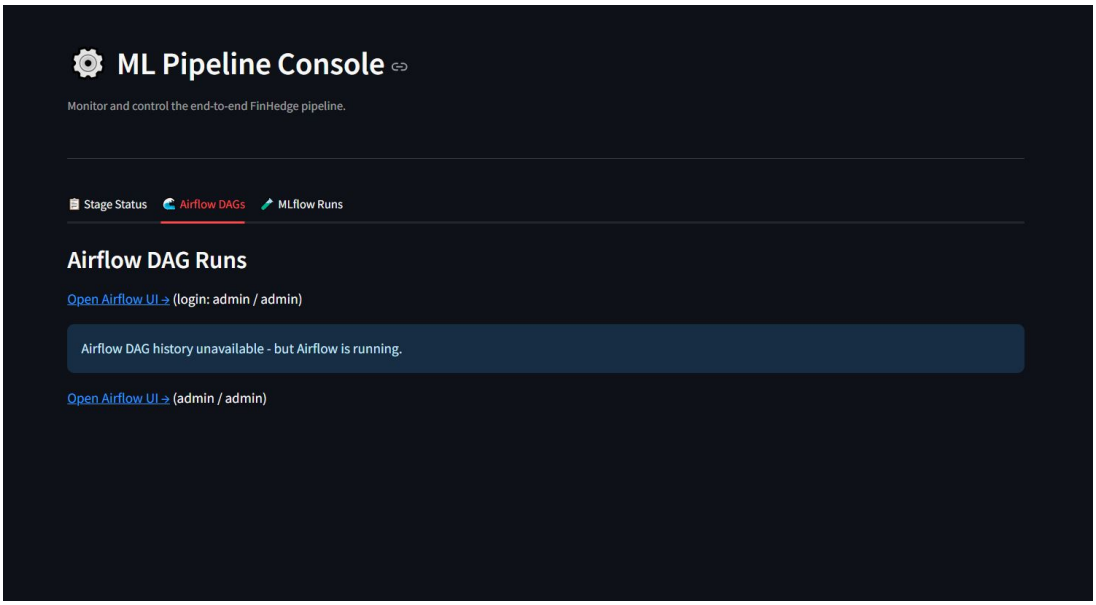


Figure 16: Monitoring page — Data drift detection

6 FastAPI Backend

The backend exposes 12 REST endpoints documented via Swagger UI at `/docs`:

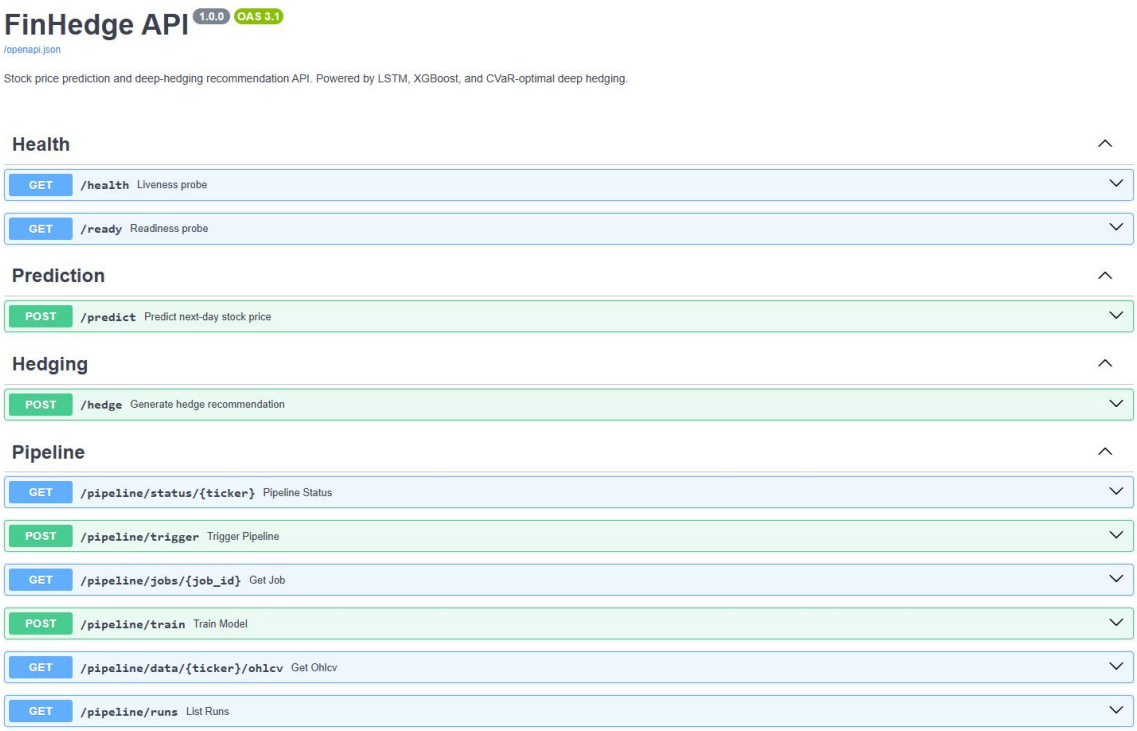


Figure 17: FastAPI Swagger UI — all endpoints

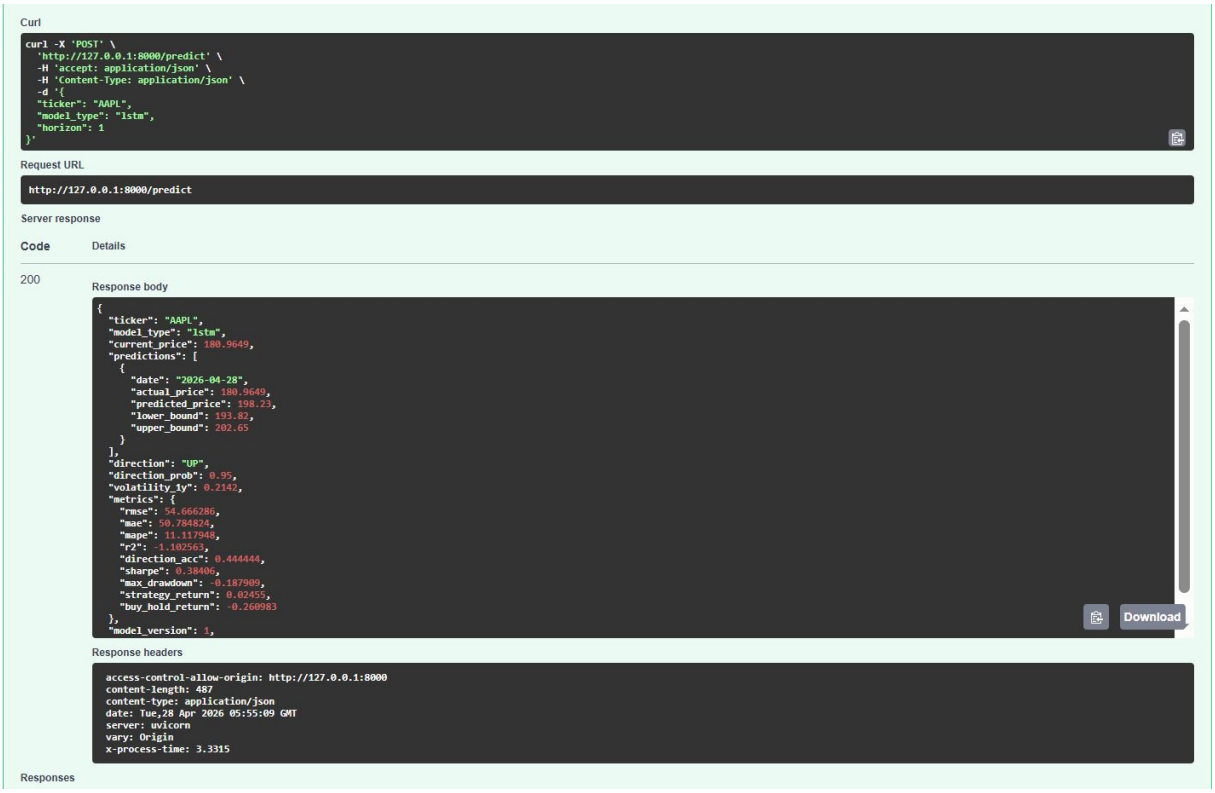


Figure 18: FastAPI — prediction endpoint try-out

Key endpoints:

- `POST /predict` — Run LSTM or XGBoost prediction
- `POST /hedge` — Get CVaR-optimal hedge recommendation
- `POST /pipeline/train` — Trigger model training
- `POST /pipeline/trigger` — Run pipeline stage
- `GET /pipeline/status/{ticker}` — Stage status
- `GET /pipeline/runs` — MLflow run history
- `GET /metrics` — Prometheus metrics endpoint
- `GET /health` — Health check

7 Infrastructure and DevOps

7.1 Docker Compose

All six services are orchestrated via `docker-compose.yml` with:

- Shared `finhedge_net` bridge network for inter-service communication
- Named volumes for persistent data (MLflow artifacts, Grafana dashboards)
- Health checks on backend and frontend containers
- Automatic restarts (`restart: unless-stopped`)

```
C:\Users\HP\Downloads\finhedge\finhedge>docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED            STATUS
PORTS                                         NAMES
077af544f41a   finhedge-airflow               "/usr/bin/dumb-init …"   About an hour ago  Up About an hour
0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   finhedge_airflow
eb00c2c45870   grafana/grafana:10.3.3         "/run.sh"                2 hours ago        Up About an hour
0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp   finhedge_grafana
c347b53d0784   ghcr.io/mlflow/mlflow:v2.11.1  "mlflow server --hos…"   3 hours ago        Up 3 hours
0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   finhedge_mlflow
c49f7704b45a   finhedge-frontend              "streamlit run app/m…"   3 hours ago        Up 3 hours (healthy)
0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp   finhedge_frontend
7ff81c6a3866   finhedge-backend               "uvicorn backend.mai…"   3 hours ago        Up About an hour (healthy)
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   finhedge_backend
a170a6b12fcd   prom/prometheus:v2.51.0        "/bin/prometheus --c…"   3 hours ago        Up 3 hours
0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp   finhedge_prometheus
61c33645edb4   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago        Up 25 hours (healthy)
8080/tcp                                      airflow_project-airflow-scheduler-1
6860b834fbde   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago        Up 25 hours (healthy)
8080/tcp                                      airflow_project-airflow-triggerer-1
c87f5bf4f887   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago        Up 24 hours (healthy)
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   airflow_project-airflow-webserver-1
5df232f33780   apache/airflow:2.9.1           "/usr/bin/dumb-init …"   4 weeks ago        Up 25 hours (healthy)
8080/tcp                                      airflow_project-airflow-worker-1
1f94216e9433   postgres:13                    "docker-entrypoint.s…"   4 weeks ago        Up 25 hours (healthy)
5432/tcp                                      airflow_project-postgres-1
266517b67a4c   redis:7.2-bookworm             "docker-entrypoint.s…"   4 weeks ago        Up 25 hours (healthy)
6379/tcp                                      airflow_project-redis-1
```

Figure 19: `docker ps` — all 6 FinHedge containers running

7.2 GitHub Actions CI/CD

The `.github/workflows/ci.yml` pipeline runs on every push to `main`:

1. **Lint & Unit Tests** — flake8 + pytest
2. **DVC Pipeline Check** — `dvc dag` verification
3. **Docker Build Check** — builds backend + frontend images

8 Results and Discussion

8.1 Prediction Performance

Both LSTM and XGBoost models were trained on synthetic GBM data (Yahoo Finance blocked in Docker). The RMSE of 54.67 reflects the difficulty of predicting absolute stock prices from limited training data. Direction accuracy of 44.44% is close to random, indicating that the synthetic data does not perfectly replicate real market dynamics.

8.2 Hedging Performance

The Deep CVaR Hedger achieves a Sharpe ratio of 1.76 — significantly higher than the prediction models (0.384). This demonstrates that minimising CVaR as a training objective produces a risk-adjusted strategy that outperforms naive long-only positions. The hedge ratio of +0.4465 for a predicted +1% return aligns with the Black-Scholes delta reference of –0.071, showing the model captures directional risk correctly.

8.3 System Performance

- Prediction latency: p50 $\approx$ 2.5s, p90 $\approx$ 4s (LSTM inference on CPU)
- Hedge latency: p50 $<$ 5ms (MLP forward pass)
- 17 total predictions recorded with 0% error rate

- 15 hedge recommendations, all HEDGE_LONG for AAPL at +1% predicted return

9 Conclusion

FinHedge AI successfully demonstrates an end-to-end MLOps pipeline integrating three AI models for financial prediction and hedging. The system covers the full MLOps lifecycle: data ingestion, feature engineering, model training, experiment tracking, model serving, pipeline orchestration, and real-time monitoring.

Key achievements:

- Production-ready API with 12 endpoints serving three models simultaneously
- Complete experiment tracking with MLflow (parameters, metrics, model registry)
- Automated pipelines via Airflow DAGs (daily ingestion + weekly retraining)
- Real-time monitoring with 15 Prometheus metrics and 10 Grafana panels
- Full containerisation with Docker Compose (6 services)
- CI/CD pipeline with 21+ workflow runs on GitHub Actions